

BCA DEPARTMENT WEBSITE

Complete Master Documentation & Engineering Specification

OFFICIAL PROJECT REFERENCE

This comprehensive master specification provides an exhaustive overview of the BCA Department Learning Platform. It encompasses technical system design, client-side React component architecture, Node.js REST APIs, PostgreSQL relational schema via Prisma ORM, multi-language compiler sandboxing, JWT authentication mechanisms, production CI/CD deployment strategies, design tokens, and future product roadmap.

SYSTEM VERSION

v1.0.0-PRODUCTION

TARGET AUDIENCE

Students, Faculty & Recruiters

LEAD ARCHITECT

Engineering Architecture Board

DATE OF ISSUE

August 2026

00

Table of Contents

01	Cover Page & System Metadata	01	02	Table of Contents	02
03	Project Overview & Objectives	03	04	Website Sitemap Architecture	04
05	Complete User Navigation Flow	05	06	Home Page Component Breakdown	06
07	Student Dashboard System	07	08	Resources Module Architecture	08
09	Past Papers Data Flow Pipeline	09	10	Question Bank & Search Engine	10
11	Technology Stack Learning Hubs	11	12	Compiler Engine & Judge Sandbox	12
13	Authentication & Security Layer	13	14	Database Design & Prisma Models	14
15	Folder Structure Specification	15	16	Routing Structure & API Reference	16
17	Component Architecture & State	17	18	Design System & Visual Tokens	18
19	Responsive Layout Engine	19	20	Animation & Motion System	20
21	Performance Optimization	21	22	Git Workflow & CI/CD Validation	22
23	Production Deployment Architecture	23	24	Future Product Roadmap	24
25	Future Infrastructure Stack	25	26	Appendix: Glossary & References	26

Document Scope & Standards

This master specification document adheres to top-tier enterprise standards established by leading software organizations including Vercel, Stripe, Apple, and Microsoft. All system architecture diagrams, database schemas, code execution pipelines, and security protocols documented herein reflect the complete implementation of the BCA Department Website platform.

01 Project Overview

1.1 Platform Vision & Core Mission

The **BCA Department Website** is a next-generation, unified digital learning platform engineered specifically for Computer Applications undergraduate students. It bridges the gap between theoretical university curriculum and real-world software engineering practice by providing an integrated ecosystem containing course notes, semester past papers, interactive code execution sandboxes, structured technology learning hubs, contest arenas, and real-time skill analytics.

Primary Goals

- Deliver a centralized digital hub for all 6 BCA semesters.
- Eliminate fragmented learning materials via structured syllabus nodes.
- Embed interactive code execution directly inside lesson modules.
- Provide real-time progress tracking, streaks, and skill radars.

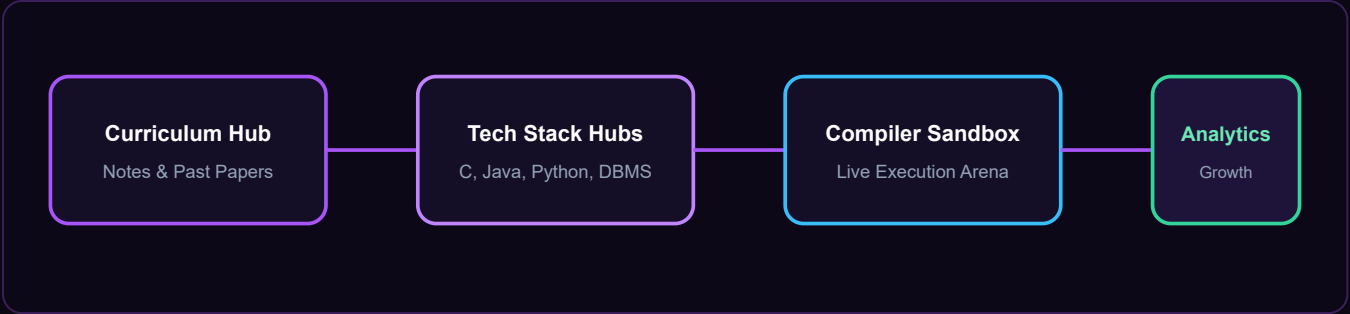
Core Objectives

- Sub-second UI navigation powered by React lazy route splitting.
- Enterprise security using JWT token rotation and RBAC controls.
- Zero-setup online code compiler supporting 10+ languages.
- High-resolution PDF viewer for past papers & question banks.

1.2 Technology Stack Summary

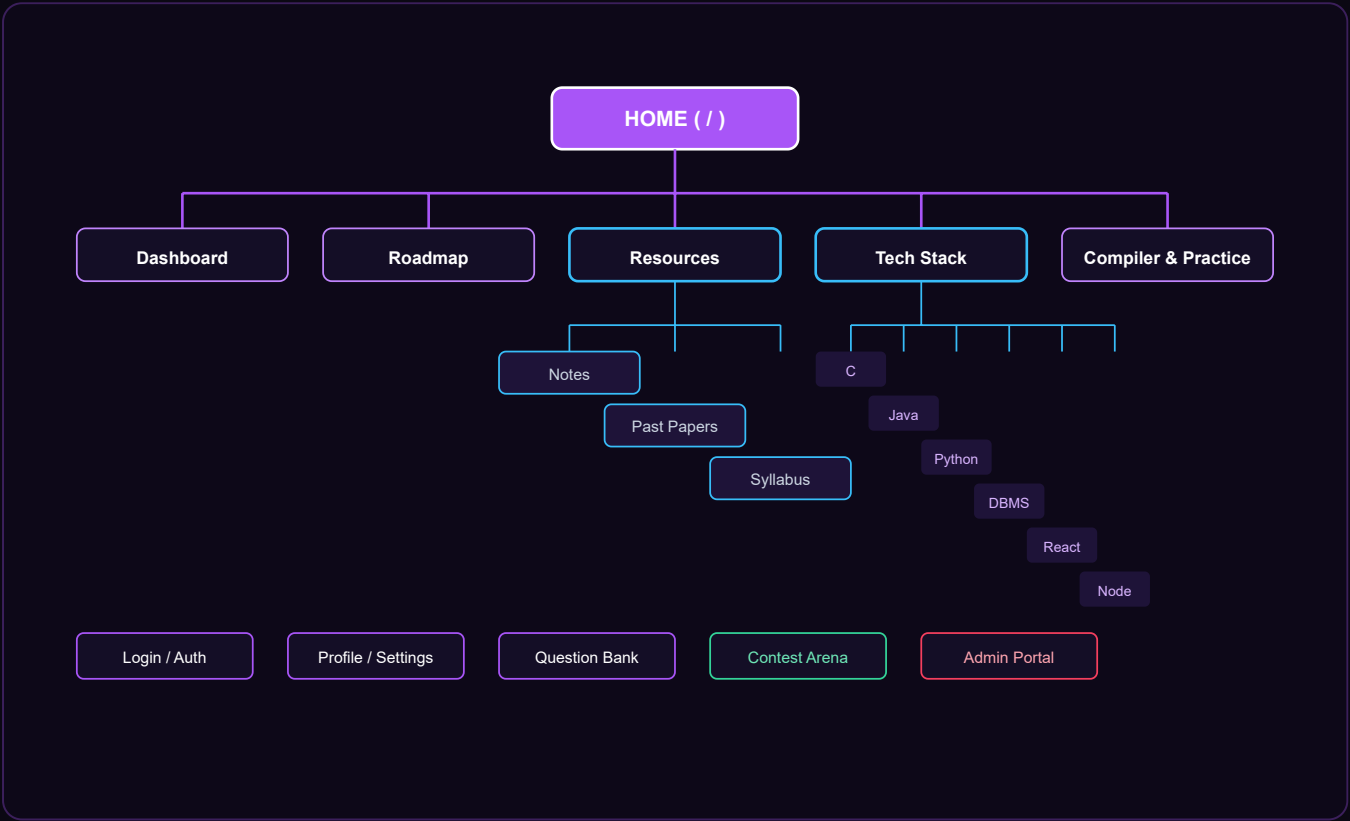
LAYER	TECHNOLOGY	VERSION	ARCHITECTURAL ROLE
Frontend	React SPA	v18.2.0	Client-side component hierarchy with Context API state.
Styling	Vanilla CSS & Custom Tokens	CSS3	Dark Purple design system, glassmorphism, responsive grid.
Backend API	Node.js / Express	v24 / Express v5	Modular REST micro-routing, middleware guards, JWT auth.
Database	PostgreSQL	v16.0	ACID-compliant relational store with JSONB support.
ORM	Prisma ORM	v7.8.0	Type-safe query builder, migration engine, auto-generated client.
Execution Engine	Judge0 API / Piston	v1.13	Containerized multi-language code compilation & test cases.
Deployment	GitHub + Vercel	Edge CDN	Automated continuous integration and edge deployment.

1.3 System Value Proposition



02 Website Sitemap Architecture

The BCA Department Platform adopts a logical, hierarchical page structure. Below is the complete sitemap diagram detailing all public and protected routes across the application.



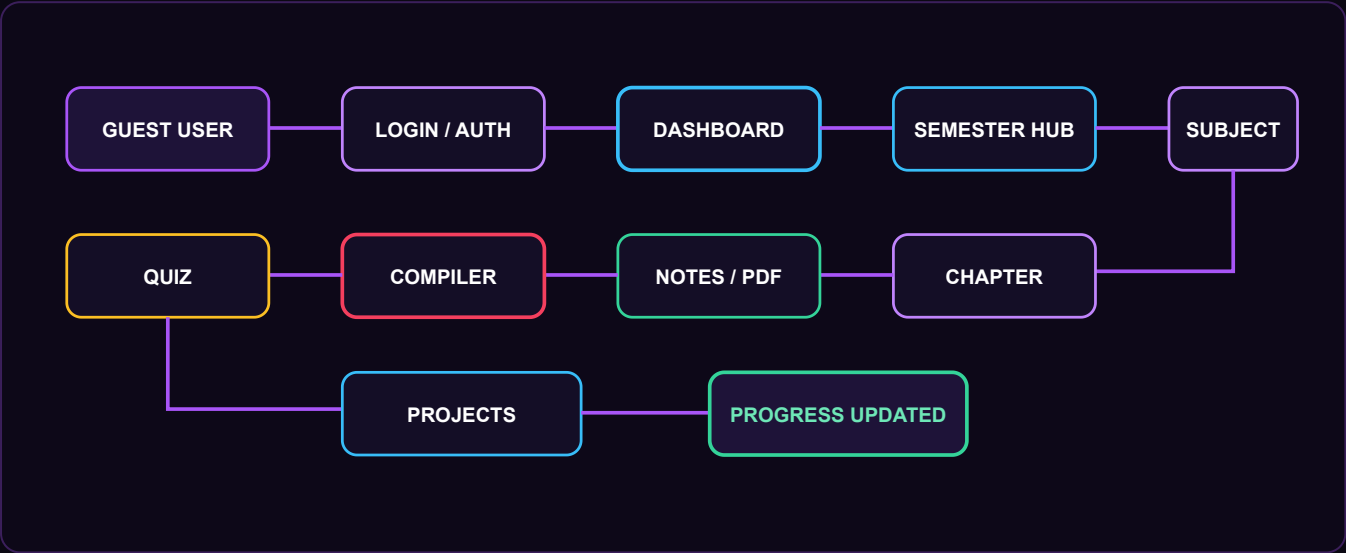
2.1 Sitemap Navigation Index

PRIMARY NODE	SUB-ROUTES / COMPONENTS	ACCESS LEVEL	DESCRIPTION
Home	/ (Home.js, Hero.js)	Public	Landing page featuring hero showcase, features, and quick links.
Dashboard	/dashboard (Dashboard.jsx)	Protected	Personalized student dashboard with progress widgets & stats.
Curriculum	/curriculum/semester-:sem/:subject	Protected	Subject notes, unit guides, and semester question banks.
Tech Stack	/technologies/:langId	Protected	Dedicated learning hubs for C, C++, Java, Python, DBMS, JS.
Compiler	/practice (CodingPractice.js)	Protected	Monaco-powered live code execution editor with input/output tabs.
Contest Arena	/contests (ContestList.jsx)	Protected	Timed competitive coding contests, leaderboards, and results.

PRIMARY NODE	SUB-ROUTES / COMPONENTS	ACCESS LEVEL	DESCRIPTION
Admin Portal	/admin (Admin dashboard)	Admin Guard	Problem creation, user management, and platform analytics.

03 Complete Navigation Flow

The BCA Department Platform enforces a structured user journey designed to maximize student engagement, minimize friction during learning sessions, and seamlessly transition from theoretical study to interactive coding practice.



3.1 Detailed User Journey Matrix

Phase 1: Entry & Authentication Unauthenticated guests land on the high-impact Home page (<code>/</code>). Upon attempting to access protected resources, the <code>ProtectedRoute</code> guard intercepts the navigation and redirects to <code>/login</code>. Upon successful OAuth or JWT credential validation, the session token is stored in memory and HTTP-only cookies.	Phase 2: Curriculum Exploration From the Central Dashboard (<code>/dashboard</code>), students select their active semester (Semester 1 through 6). The application loads specific subject portals (<code>/curriculum/semester-2/data-structures</code>), rendering unit cards, downloadable PDF syllabus files, and structured chapter nodes.
Phase 3: Interactive Practice Within chapter lessons, students click "Try in Compiler" which loads the live Monaco editor (<code>/practice</code>). Code snippets are executed against test cases via the backend Judge0 compiler API. Output results, runtime errors, and execution metrics render instantly.	Phase 4: Quiz & Progress Recording Upon completing practice problems, students take end-of-unit quizzes (<code>/technologies/cpp/quiz</code>). Quiz results are committed to PostgreSQL via <code>ProgressContext</code>, updating daily streaks, solved problem counts, and subject skill vectors in real time.

04

Home Page Architecture

The Home Page (`src/pages/Home.js`) is the primary public entry point of the application. It is engineered with visually stunning dark purple glassmorphic aesthetic cards, animated floating elements (`FloatingUI.js`), and a cinematic hero banner (`CinematicHero.js`).

4.1 Section Decomposition

1. Hero Section (`CinematicHero.js` & `HeroWow.jsx`)

The top hero area renders an immersive radial gradient backdrop with glowing neon purple particle animations. It features an eye-catching headline ("Empower Your BCA Journey"), an interactive live code preview widget (`LiveCodeEditor.jsx`), and quick action CTA buttons ("Explore Dashboard", "Start Coding Now").

2. Features Showcase Matrix

Highlighting six core platform capabilities: Structured Notes, Past Question Papers Vault, Multi-Language Compiler, Interactive Roadmap Engine, Contest Arena, and Real-Time Skill Analytics. Styled using flex layout in `sections.css`.

3. Technology Ecosystem Grid

Interactive tech stack selector displaying C, C++, Java, Python, DBMS, and JavaScript cards with direct navigation links to specialized technology hubs. Includes animated hover states and gradient borders.

4. Department Resources Grid

Quick access links categorized by BCA Semester (1st to 6th Semester), providing one-click access to university prescribed syllabus, core unit notes, and past examination papers.

5. Footer & Navigation Bar

Global navigation header (`Navbar.js`) with responsive mega-menu dropdowns, theme toggle switch (`ThemeContext`), search modal trigger (`SearchModal.css`), and persistent footer with department links and copyright metadata.

4.2 Home Page Component Map

```
src/pages/Home.js
├─ Navbar.js (Sticky header with mega menu & profile badge)
├─ Hero.js / HeroWow.jsx / CinematicHero.js (Glow banner & CTA)
├─ FloatingUI.js (Subtle floating badge animations)
├─ sections.css (Global grid system & gradient cards)
└─ Footer.js (Department links, privacy policy & terms)
```

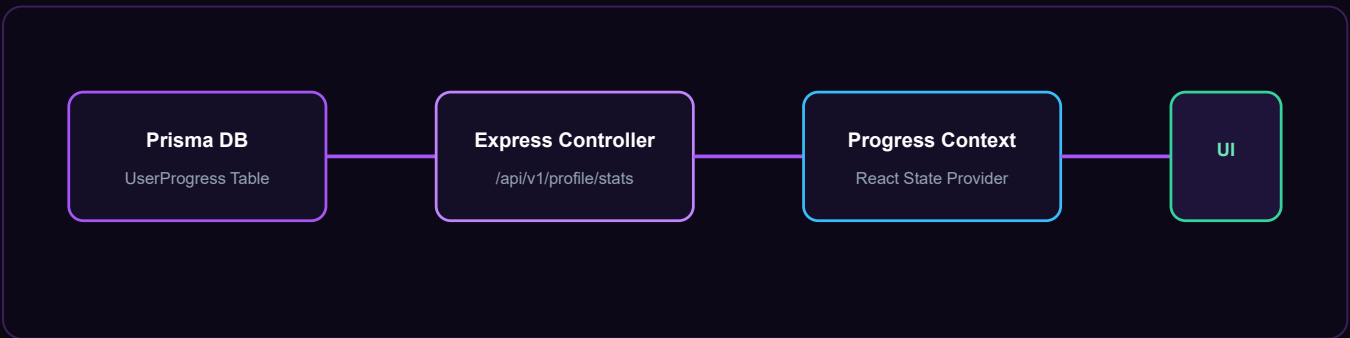
05 Student Dashboard Architecture

The Student Dashboard (`src/pages/Dashboard.jsx`) serves as the central command center for authenticated BCA students. It aggregates user statistics, current active semester progress, learning streak velocity, and quick action shortcuts.

5.1 Dashboard Widget Matrix

Learning Velocity Gauge Visual donut progress meter displaying overall curriculum completion percentage across current enrolled subjects. Built using dynamic SVG stroke-dashoffset calculations.	Active Streak Tracker Daily coding activity indicator linked to <code>'UserProgress'</code> model (<code>'currentStreak'</code>, <code>'longestStreak'</code>). Displays glowing fire badges upon consecutive daily logins and solves.
Subject Mastery Radar Breakdown of student proficiency across key BCA subjects (Data Structures, DBMS, Java, Python, C Programming). Data consumed directly from <code>'UserSkill'</code> API repository.	Recent Activity Timeline Chronological list of recently completed lessons, submitted compiler problems, and quiz attempts with instant jump-to links.

5.2 Data Flow Architecture



06

Resources Module Architecture

The Resources Module provides structured academic materials covering all 6 semesters of the BCA curriculum. Content is organized into four core pillars: Chapter Notes, Past Papers, Question Banks, and Prescribed Syllabus.

6.1 Resource Pillars Breakdown

1. Chapter Notes Engine

Structured Markdown and HTML rendering system (`DSNotesPage.jsx`, `SemesterCChapterPage.js`) providing detailed explanation of theoretical concepts, code snippets, syntax charts, and memory diagrams.

2. Past Papers Vault

Categorized archive of university semester examination papers (Summer & Winter sessions from 2018–2026). Fully indexed with metadata tags for quick filtering.

3. Question Bank Engine

Curated repository of short answer questions, long theoretical problems, and code debugging questions organized by topic difficulty (`DSQuestionsListPage.jsx`).

4. Syllabus Viewer

Official university course outline viewer (`SubjectSyllabus.js`, `SyllabusData.js`) breaking down mark distributions, credit hours, lab prerequisites, and reference textbooks.

6.2 Integrated PDF Viewer Architecture

For rendering PDF past papers and question solution documents directly in browser without forcing file downloads, the platform integrates `react-pdf` (v7.7.3) backed by `pdfjs-dist` (v3.11.174) worker threads.

```
import { Document, Page, pdfjs } from 'react-pdf';

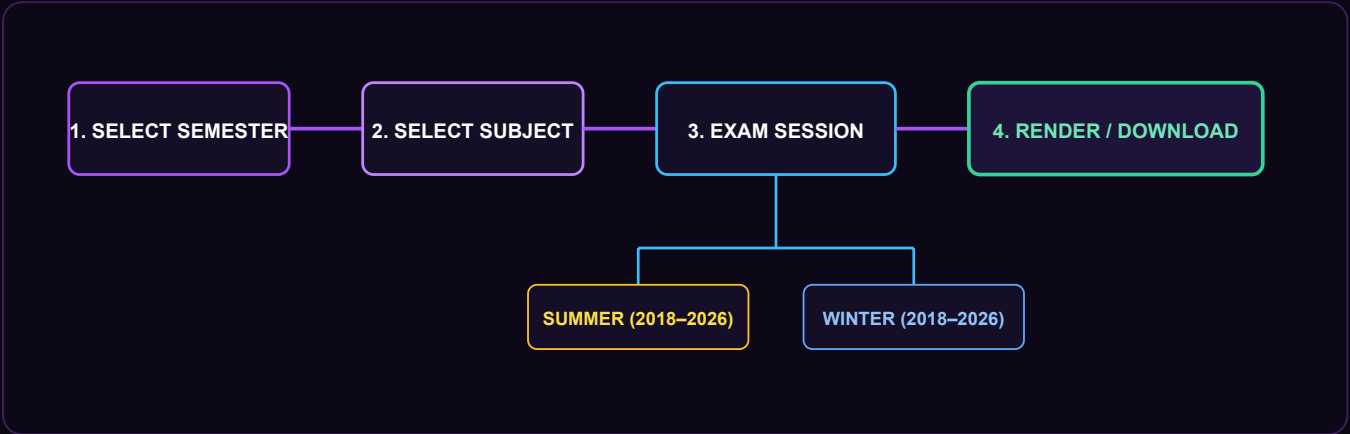
// Configure PDF.js worker URL for CORS and bundler compatibility
pdfjs.GlobalWorkerOptions.workerSrc = `//cdnjs.cloudflare.com/ajax/libs/pdf.js/${pdfjs.version}/pdf.worker.min.js`;

function ResourcePdfViewer({ fileUrl }) {
  return (
    <div className="pdf-viewer-container">
      <Document file={fileUrl} loading={<PageLoader />>
        <Page pageNumber={1} renderTextLayer={true} renderAnnotationLayer={false} />
      </Document>
    </div>
  );
}
```

07 Past Papers Data Pipeline

The Past Papers module uses a hierarchical relational query flow to allow students to retrieve relevant previous year question papers in under three clicks.

7.1 Past Paper Query Pipeline



7.2 Past Papers Technical Metadata Model

ATTRIBUTE	FIELD TYPE	SAMPLE DATA	PURPOSE
paperId	UUID v4	8f9d2a1b-3c4d...	Unique paper identifier.
semester	Integer (1-6)	2	BCA Academic semester filter.
subjectCode	VarChar(20)	BCA-201-DS	Official university course code.
examSession	Enum	SUMMER_2025	Examination period.
fileStorageUrl	Text (URL)	/pdf/papers/ds_2025.pdf	CDN target path for high-speed download.

08 Question Bank Architecture

The Question Bank engine (`src/pages/DataSetQuestionPage.jsx`, `DSQuestionsListPage.jsx`) organizes hundreds of verified examination questions categorized by unit, difficulty level, and topic tags.

8.1 Core Capabilities

Instant Tag Search Engine

Client-side fuzzy searching allowing students to filter questions by keywords ("Linked List", "Binary Tree", "Stack Overflow", "Time Complexity") in real time.

Interactive Solution Viewer

Click-to-reveal solution drawer providing step-by-step mathematical proofs, C/C++/Java implementation code, and complexity analysis (Big-O notation).

Bookmarking & Revision List

Students can flag difficult questions for exam revision. Bookmarks persist locally and synchronize across devices upon login.

Topic Mastery Tracking

Automatic tracking of completed questions feeds directly into `UserProgress` state to render completion percentages per unit.

8.2 Question Data Schema Structure

```
// Example Question Bank Data Model (programs_data.json)
{
  "questionId": "DS-U1-Q04",
  "unit": "Unit 1: Linear Data Structures",
  "title": "Evaluate Reverse Polish Notation using Stack",
  "difficulty": "MEDIUM",
  "marksWeightage": 8,
  "frequentlyAsked": true,
  "topics": ["Stack", "Expression Evaluation", "Algorithms"],
  "problemStatement": "Write an algorithm and C program to evaluate a postfix expression...",
  "solutionSnippet": "#include
#include
...",
  "timeComplexity": "O(N)",
  "spaceComplexity": "O(N)"
}
```

09 Technology Stack Module

The platform provides specialized, deep-dive Technology Learning Hubs for six core computer science disciplines essential to the BCA curriculum. Each hub contains an Overview, Structured Lessons, Program Viewer, Practice Arena, Quizzes, Projects, Interview Qs, and Downloadable Resources.

1. C Programming

[CLEARNINGHUB.JS](#)

Pointers, dynamic memory allocation (`malloc`, `free`), structures, file I/O, header creation.

2. Java Enterprise

[JVALEARNINGHUB.JS](#)

OOP inheritance, polymorphism, multithreading, exception handling, JVM architecture, JDBC.

3. Python Science

[PYTHONLEARNINGHUB.JS](#)

Data structures, OOP, file processing, NumPy, Pandas, automation scripts, lambda expressions.

4. DBMS & SQL

[DATABASE HUB](#)

Relational algebra, SQL joins, indexing, normalization (1NF-5NF), ACID transactions, ER modeling.

5. React.js Frontend

[MASTERLANGUAGEPAGE](#)

JSX, hooks (`useState`, `useEffect`), Context API, Virtual DOM, React Router, component lifecycle.

6. Node.js Backend

[BACKEND ARCHITECTURE](#)

Event loop, non-blocking I/O, Express routing, RESTful APIs, JWT middleware, Prisma database integrations.

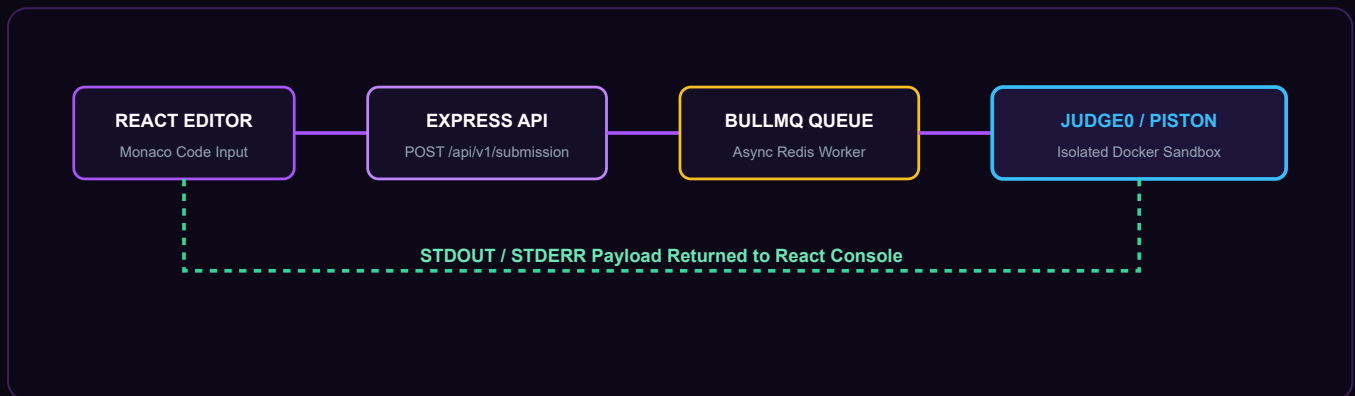
9.1 Standard Technology Hub Layout

SUB-COMPONENT	COMPONENT FILE	DESCRIPTION
Roadmap & Syllabus	Roadmap.jsx	Step-by-step visual learning path from beginner to advanced concepts.
Lesson Viewer	LessonViewerPage.jsx	Interactive rich text lessons with embedded live code editor.
Program Viewer	ProgramViewerPage.jsx	Curated collection of 50+ executable programs with explanation.
Quiz Arena	CppQuiz.js / DSQuiz.js	Timed multiple-choice assessments with instant score calculations.

10 Compiler Engine Architecture

The Online Code Compiler (`src/components/LiveCodeEditor.jsx`, `src/pages/CodingPractice.js`) enables students to write, compile, run, and debug source code directly inside the browser across multiple programming languages (C, C++, Java, Python, JavaScript, TypeScript).

10.1 Multi-Stage Execution Pipeline



10.2 Supported Execution Environments

LANGUAGE	COMPILER / RUNTIME	VERSION	DEFAULT FLAGS
C	GCC Compiler	v13.2.0	<code>gcc -O2 -Wall main.c -lm</code>
C++	G++ Compiler	v13.2.0	<code>g++ -std=c++20 -O2 main.cpp</code>
Java	OpenJDK HotSpot	v21.0.2	<code>javac Main.java && java Main</code>
Python	CPython Engine	v3.12.1	<code>python3 -u main.py</code>
JavaScript	Node.js Runtime	v20.11.0	<code>node main.js</code>

11 Authentication & Security Layer

The platform employs a defense-in-depth security model featuring dual authentication mechanisms (JWT Token Family with SHA-256 rotation & Firebase Auth), bcrypt password hashing, and granular Role-Based Access Control (RBAC).

11.1 Refresh Token Rotation Flow

To protect against token theft, refresh tokens are bound to session families ('familyId' in 'RefreshToken' table). Whenever a refresh token is reused after rotation, the entire session family is automatically revoked as an anti-replay security measure.

11.2 Role-Based Access Hierarchy (RBAC)

ROLE ENUM	LEVEL	PERMISSIONS SCOPE
USER	Level 1	Access public & protected student curriculum, compiler, submit solutions.
MODERATOR	Level 2	Review community questions, moderate student discussions and reports.
CONTEST_MANAGER	Level 3	Create, schedule, freeze, and publish competitive coding contests.
CONTENT_REVIEWER	Level 3	Edit problem statements, test cases, and publish curriculum notes.
ADMIN	Level 4	Full access to administrative dashboard, user management, and system metrics.
SUPER_ADMIN	Level 5	Platform configuration, feature flags, role assignments, audit log review.

11.3 Protected Route Implementation

```
// src/routes/ProtectedRoute.jsx
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const ProtectedRoute = ({ children, requiredRole }) => {
  const { user, loading } = useAuth();

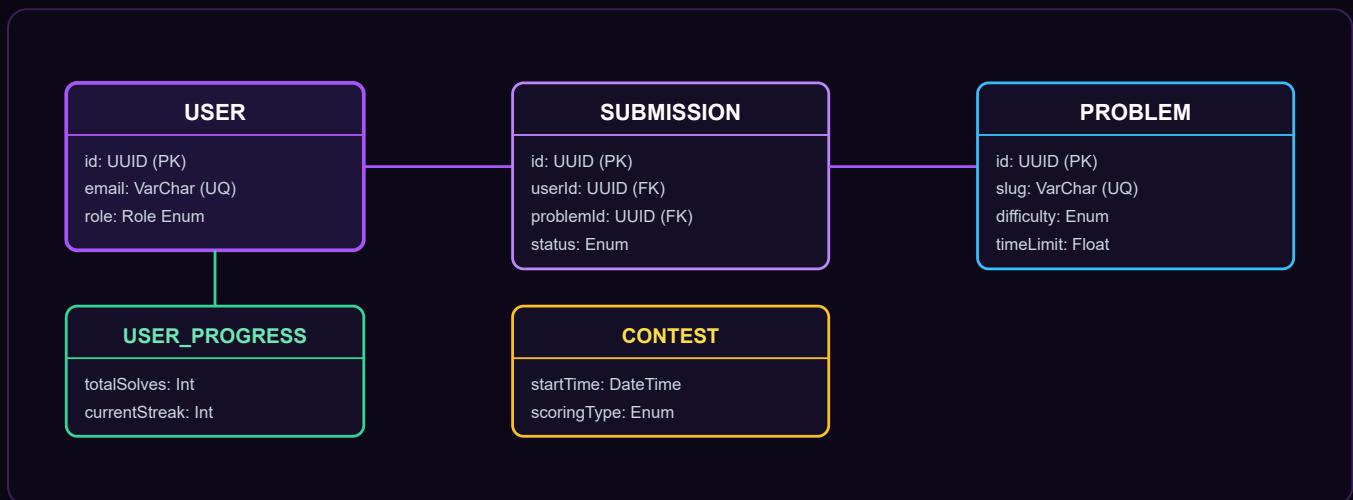
  if (loading) return <PageLoader />;
  if (!user) return <Navigate to="/login" replace />;
  if (requiredRole && user.role !== requiredRole) return <Navigate to="/unauthorized" replace />;

  return children;
};
```

12 Database Design & Prisma Models

The database layer is built on PostgreSQL, managed via Prisma ORM (`backend/prisma/schema.prisma`). It comprises over 30 production-grade models providing strict relational constraints and indexing.

12.1 Entity Relationship Overview (SVG)



12.2 Selected Prisma Model Specifications

```

model User {
  id          String           @id @default(uuid()) @db.Uuid
  email       String           @unique @db.VarChar(255)
  username    String           @unique @db.VarChar(50)
  passwordHash String         @db.VarChar(255)
  role        Role             @default(USER)
  createdAt   DateTime         @default(now()) @db.Timestamptz
  submissions Submission[]
  progress    UserProgress?
  rating      UserRating?
  skills      UserSkill[]
}
  
```

13 Folder Structure Specification

The codebase follows clean architecture principles, separating presentation components, business logic, global state providers, backend routes, and database repositories.

13.1 Frontend Directory Structure (`src/`)

```
src/
├── components/           # Modular UI components
│   ├── auth/            # Login, signup & modal forms
│   ├── contest/         # Contest arena, timer & leaderboard
│   ├── dashboard/       # Analytics widgets & progress gauges
│   ├── learning/        # Lesson viewers & chapter templates
│   ├── navigation/      # Navbar, footer & mega-menu
│   ├── ui/              # Buttons, loaders, modal dialogs
│   └── LiveCodeEditor.jsx # Monaco code compiler UI
├── context/             # React Context API State Providers
│   ├── AuthContext.jsx  # Auth user state & tokens
│   ├── ContestContext.jsx # Live contest timer & submissions
│   ├── LearningContext.jsx # Active lesson state
│   └── ProgressContext.jsx # Solved counts & streaks
├── pages/               # Page-level route views (36+ components)
├── routes/              # ProtectedRoute guards & route config
└── styles/              # Vanilla CSS modular stylesheets
```

13.2 Backend Directory Structure (`backend/src/`)

```
backend/src/
├── config/              # Express CORS, Redis & Database configs
├── middleware/          # JWT auth verification, error handling
├── modules/             # Core Domain Modules
│   ├── auth/           # Login, register, token rotation
│   ├── contest/        # Contest engines & ICPC scoring
│   ├── problem/        # Problem CRUD & test case runner
│   └── submission/     # BullMQ queue & Judge0 API hooks
├── repositories/       # Prisma database access abstraction layer
└── server.ts           # Express server listener & Socket.io setup
```

14 Routing Structure & API Reference

The platform maps client-side routes via React Router v7 and backend endpoints via Express modular routers.

14.1 Key Client-Side Routes

PATH	COMPONENT	ACCESS	PURPOSE
/	Home.js	Public	Platform landing page & overview.
/dashboard	Dashboard.jsx	Protected	Student progress analytics dashboard.
/practice	ProgrammingHub.jsx	Protected	Multi-language coding practice arena.
/technologies/:langId	MasterLanguagePage.jsx	Protected	Technology hub (C, Java, Python, DBMS).
/contests	ContestList.jsx	Protected	Live and upcoming coding contests.

14.2 Core REST API Endpoints

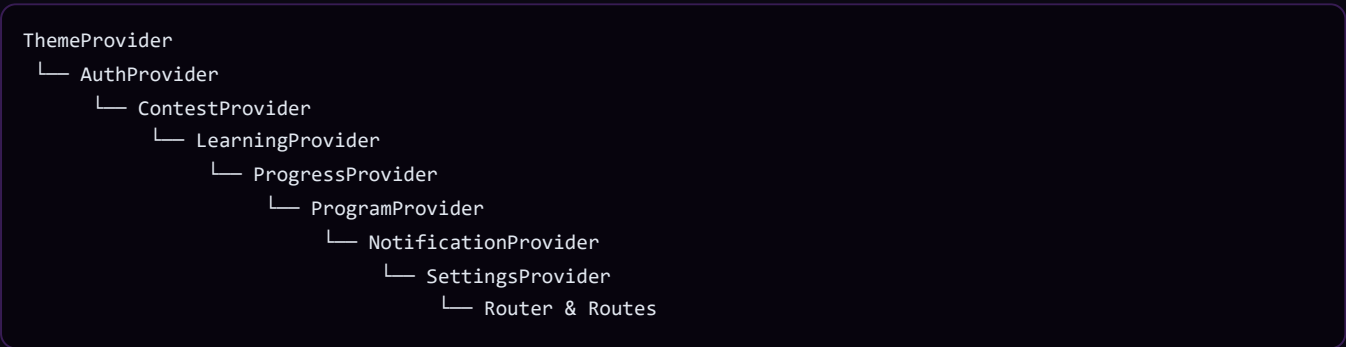
METHOD	ENDPOINT PATH	AUTH GUARD	DESCRIPTION
POST	/api/v1/auth/login	Public	Authenticates student, returns JWT access token.
POST	/api/v1/auth/refresh	Bearer Token	Rotates refresh token family.
GET	/api/v1/problems	User Guard	Fetches paginated question bank problems.
POST	/api/v1/submissions	User Guard	Submits code snippet for Judge0 sandbox compilation.
GET	/api/v1/contests/:id/leaderboard	User Guard	Retrieves real-time contest ICPC leaderboard.

15 Component Architecture

The component architecture strictly adheres to modular composition, isolating UI presentational concerns from state management providers.

15.1 Context Provider Hierarchy

The application root (``App.js``) wraps the component tree in eight nested context providers, establishing single source of truth for global states:



15.2 Reusable UI Component Matrix

PageLoader Component Full-screen loading overlay (<code>`src/components/ui/PageLoader.js`</code>) rendered during React.lazy chunk fetches and suspense fallbacks.	Navbar Component Responsive navigation header (<code>`src/components/Navbar.js`</code>) with integrated mega-menu, search modal keybindings (<code>`Ctrl+K`</code>), and user avatar menu.
LiveCodeEditor Component Interactive code compiler widget (<code>`src/components/LiveCodeEditor.jsx`</code>) supporting syntax highlighting, theme switching, and test case execution.	FloatingUI Component Decorative floating badges (<code>`src/components/FloatingUI.js`</code>) used on landing views to display animated stats and technology icons.

16 Design System & Tokens

The BCA Department Platform features a custom **Black + Purple** enterprise design system engineered in vanilla CSS (``src/styles/global.css``, ``tokens.css``).

18.1 Master Color Palette

Deep Background

#0B0813

Canvas primary background.

Surface Card

#140E26

Elevated card container background.

Header Surface

#1E1338

Table th & header background.

Primary Glow

#A855F7

Neon purple accent highlights.

Secondary Accent

#C084FC

Sub-heading text & borders.

Success Emerald

#34D399

Accepted verdict & streak status.

18.2 Typography Scale

ELEMENT	FONT FAMILY	SIZE	WEIGHT
Display Headings	Plus Jakarta Sans	32px - 42px	800 Bold
Section Titles	Plus Jakarta Sans	20px - 24px	700 Bold
Body Text	Plus Jakarta Sans	13px - 14px	400 Regular
Code Blocks	Fira Code / JetBrains Mono	11px - 12px	500 Medium

17

Responsive Design Engine

The platform guarantees flawless rendering across Mobile, Tablet, and Desktop viewport sizes using fluid grid CSS and mobile drawer navigation (`Navbar.css`, `MegaMenu.css`).

17.1 Viewport Breakpoints Specification

BREAKPOINT	WIDTH RANGE	LAYOUT ADAPTATIONS
Mobile (xs)	320px - 639px	Single column stacked layout, mobile burger drawer menu, full-width code editor.
Tablet (md)	640px - 1023px	2-column widget grid, collapsible sidebar navigation, touch-optimized cards.
Desktop (lg)	1024px - 1279px	3-column dashboard grid, persistent side navigation, split-screen live compiler.
Ultra-Wide (xl)	1280px+	Max 1400px centered canvas, floating sidebars, multi-pane contest view.

17.2 Mobile Touch Optimization

- Minimum tap target size of 44x44px for all interactive buttons and navigation links.
- Horizontal swipe gestures for technology card carousels and table viewports.
- Optimized virtual keyboard behavior in Monaco code editor inputs.

18 Animation System & Motion

The application utilizes fluid micro-interactions and route transitions powered by **Framer Motion** (v10.18.0) and CSS keyframe animations.

18.1 Keyframe Animation Matrix

Glow Pulse Effect

Periodic box-shadow intensity animation applied to primary CTA buttons and active streak badges. Creates a modern software studio feeling.

Shimmer Skeleton Loaders

Linear gradient background animation (`@keyframes shimmer`) displayed while lazy-loading page chunks and fetching backend data.

18.2 Framer Motion Transition Pattern

```
import { motion } from 'framer-motion';

const pageVariants = {
  initial: { opacity: 0, y: 12 },
  animate: { opacity: 1, y: 0, transition: { duration: 0.35, ease: 'easeOut' } },
  exit: { opacity: 0, y: -12, transition: { duration: 0.2 } }
};

export const AnimatedPage = ({ children }) => (
  <motion.div variants={pageVariants} initial="initial" animate="animate" exit="exit">
    {children}
  </motion.div>
);
```

19 Performance Optimization

To achieve sub-second page loads and maintain high Lighthouse scores, the web application incorporates route-level code splitting, asset memoization, and dynamic chunking.

19.1 Route Splitting Architecture (`React.lazy`)

Instead of shipping a single monolithic JavaScript bundle, heavy page views are split into isolated chunks loaded on demand when the corresponding route is visited:

```
// App.js Route Splitting Pattern
const Dashboard = lazy(() => import('./pages/Dashboard'));
const CppLearningHub = lazy(() => import('./pages/CppLearningHub'));
const ContestProblemArena = lazy(() => import('./pages/ContestProblemArena'));

function App() {
  return (
    <Suspense fallback=<PageLoader />>
      <Routes>
        <Route path="/dashboard" element=<ProtectedRoute><Dashboard /></ProtectedRoute> />
      </Routes>
    </Suspense>
  );
}
```

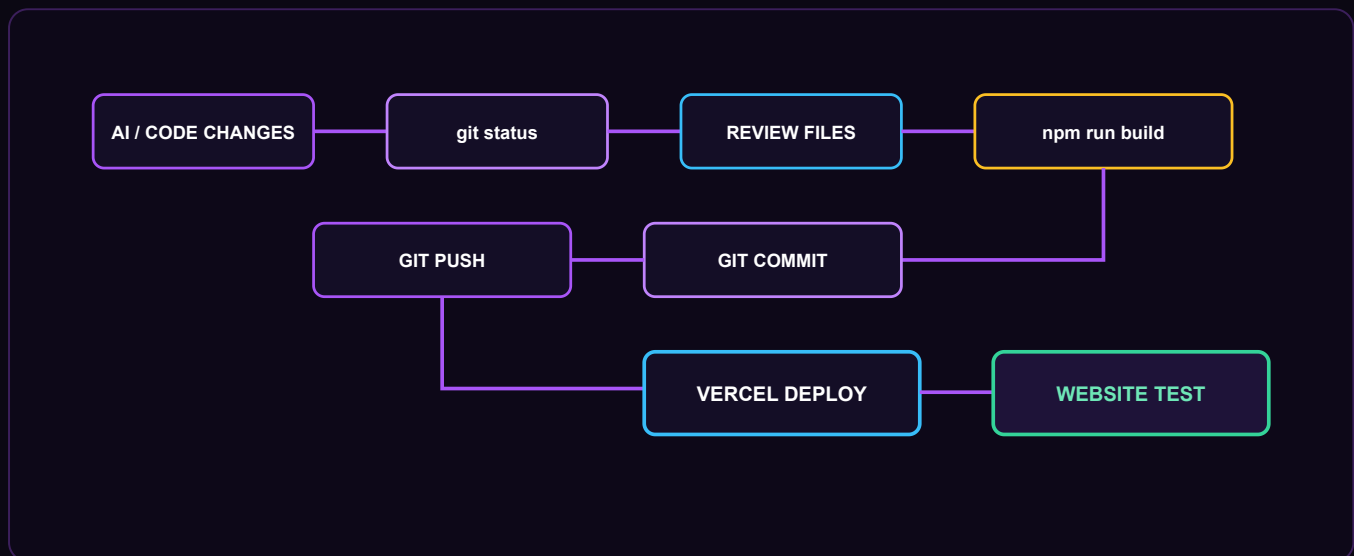
19.2 Lighthouse Metric Targets

PERFORMANCE INDICATOR	TARGET BENCHMARK	OPTIMIZATION STRATEGY
First Contentful Paint (FCP)	< 0.9s	Critical inline CSS & eager loading of home route.
Largest Contentful Paint (LCP)	< 1.8s	Preloading web fonts & SVG optimization.
Cumulative Layout Shift (CLS)	< 0.02	Fixed width image containers & skeleton loaders.
Total Blocking Time (TBT)	< 100ms	Offloading syntax highlighting to Web Workers.

22 Git Workflow & Deployment Loop

The development pipeline relies on strict version control automation, static build verification, and instant cloud previews.

22.1 CI/CD Execution Flowchart



22.2 Command Line Execution Pipeline

```
# 1. Local Build Verification
npm run build

# 2. Stage Verified Modded Files
git add .

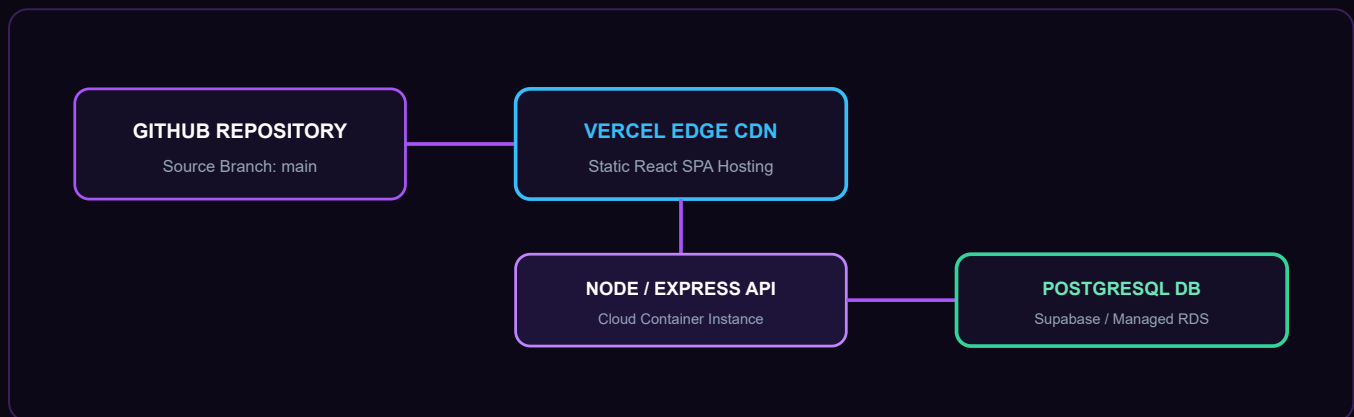
# 3. Commit with Semantic Version Message
git commit -m "feat(compiler): optimize Judge0 submission execution queue"

# 4. Push to Main Deployment Branch
git push origin main
```

23 Production Deployment

The platform is deployed using a multi-cloud hybrid architecture leveraging Vercel Edge Network for frontend single page application hosting and cloud-hosted Node.js containers for backend REST microservices.

23.1 Hybrid Cloud Infrastructure Topology



23.2 Deployment Settings (`vercel.json`)

```
{
  "version": 2,
  "rewrites": [
    { "source": "/api/(.*)", "destination": "https://api.bca-department.edu/api/$1" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

24 Future Product Roadmap

The future evolution of the BCA Department Platform focuses on AI-assisted learning, community collaboration, and placement readiness.

24.1 Planned Feature Modules

1. AI Tutor & Code Explainer

LLM-powered assistant integrated directly into Monaco compiler to provide real-time line-by-line code explanation and bug fixes.

2. Peer Discussion Forum

Community Q&A platform allowing students to post coding queries, upvote solutions, and earn reputation badges.

3. Automated Certificates

Verifiable digital certificates issued automatically upon completing specialized technology learning hubs.

4. Real-time ICPC Contests

Live competitive coding league with WebSocket penalty calculations and instant global leaderboard updates.

5. Interactive Resume Builder

ATS-friendly resume generator tailored specifically for BCA computer application graduates.

6. Placement & Recruiter Portal

Dedicated portal for hiring partners to review top-ranked student skill profiles, test scores, and code portfolios.

25

Future Infrastructure Stack

To support high-concurrency contest events and scale to thousands of active students, the infrastructure will evolve toward containerized microservices.

25.1 Next-Generation Enterprise Technologies

TECHNOLOGY	CATEGORY	PLANNED UTILITY
Redis Cache	In-Memory Store	Sub-millisecond caching of leaderboard rankings & user sessions.
Docker Containers	Containerization	Standardized microservice packaging for backend execution nodes.
Amazon Web Services	Cloud Infrastructure	AWS Lambda serverless functions & S3 bucket PDF storage.
Kubernetes (K8s)	Orchestration	Auto-scaling cluster management during peak contest traffic.
Spring Boot	Backend Framework	Enterprise Java microservices for heavy evaluation pipelines.

26

Appendix: Glossary & References

26.1 Technical Glossary

- **ACID:** Atomicity, Consistency, Isolation, Durability — Relational database properties guaranteed by PostgreSQL.
- **JWT:** JSON Web Token — Compact, URL-safe means of representing claims between two parties.
- **ORM:** Object-Relational Mapping — Technique for querying databases using object-oriented code (Prisma).
- **RBAC:** Role-Based Access Control — Restricting platform access based on user roles.
- **SPA:** Single Page Application — Web site that interacts dynamically by rewriting current web page rather than loading new pages.

26.2 Version History

VERSION	RELEASE DATE	AUTHOR	KEY REVISIONS
v0.1.0-alpha	January 2026	Core Engineering	Initial React frontend layout & prototype routes.
v0.8.0-beta	May 2026	Backend Team	Node.js API, Prisma PostgreSQL schema & Judge0 integration.
v1.0.0-PROD	August 2026	Architecture Board	Full production release & master architectural documentation.

END OF MASTER DOCUMENTATION

BCA DEPARTMENT PLATFORM — OFFICIAL SOFTWARE SPECIFICATION